

FitLog — Annotations Reference & Viva Study Guide

Project Leap — Java & DBMS Examination (Regulation 2023) | Course: U28CS491
Student Reg No: 722825243194 | Sri Eshwar College of Engineering, Coimbatore

1. Spring Boot Core & Configuration

Annotation	Technical Function	Where Used	Viva / Examiner Tip
@SpringBootApplication	Bootstraps and configures the Spring Boot application context. Combines <code>@Configuration</code> (enables Java-based config), <code>@EnableAutoConfiguration</code> (auto-configures Tomcat, JPA, Jackson), and <code>@ComponentScan</code> (scans <code>com.fitlog</code> package for components).	FitLogApplication.java	The central launchpad of the application. Tell the examiner it eliminates dozens of lines of XML by automatically finding all <code>@RestController</code> , <code>@Service</code> , and <code>@Repository</code> classes.
@Configuration	Marks a class as a source of bean definitions. Tells Spring IoC container that the class contains methods annotated with <code>@Bean</code> .	OpenApiConfig.java	Used for programmatic configuration. In FitLog, it defines our custom Swagger / OpenAPI title, version, and student metadata.
@Bean	Indicates that a method instantiates, configures, and initializes a new object managed as a singleton bean by the Spring IoC container.	OpenApiConfig.java (customOpenAPI)	When you need a third-party class (like OpenAPI) in your Spring container, you write a method annotated with <code>@Bean</code> returning that instance.
@Value	Injects externalized values from <code>application.properties</code> or system environment variables into Java fields at runtime.	ChatServiceImpl.java	Prevents hardcoding sensitive credentials or endpoints in Java code. For example, <code>@Value("\${groq.api.key}")</code> injects the Groq API key.

2. Stereotypes & Architecture

Annotation	Technical Function	Where Used	Viva / Examiner Tip
@RestController	Specialized version of <code>@Controller</code> that combines <code>@Controller</code> and <code>@ResponseBody</code> . Every handler method returns data directly serialized into JSON rather than rendering an HTML view template.	UserController, WorkoutController, MealController, GoalController, SummaryController, DashboardController, ChatController	Essential for pure REST APIs. Tell the examiner it tells Spring: 'Do not look for a JSP/Thymeleaf HTML page; write the return object straight into the HTTP response body as JSON.'
@Service	Marks a class as a business service component in the business logic layer. Spring automatically discovers it during component scanning and manages it as a singleton bean.	UserServiceImpl, WorkoutServiceImpl, MealServiceImpl, GoalServiceImpl, SummaryServiceImpl, DashboardServiceImpl, ChatServiceImpl	Separates business logic from HTTP presentation and database access. All business rules (e.g. non-negative calories) are strictly enforced inside <code>@Service</code> classes.
@Repository	Marks a data access interface/class as a Spring Data repository component. Enables automatic implementation generation and persistence exception translation.	UserRepository, WorkoutRepository, MealRepository, GoalRepository	Spring Data JPA automatically creates proxy implementations of these interfaces at runtime with complete CRUD methods (save, findById, findAll, delete).
@RestControllerAdvice	Defines a global, centralized exception interceptor for all <code>@RestController</code> classes. Intercepts exceptions and transforms them into standardized JSON error responses.	GlobalExceptionHandler.java	Crucial for clean enterprise architecture. Instead of letting ugly 500 stack traces reach the client, <code>@RestControllerAdvice</code> converts Java exceptions into neat HTTP 400/404 JSON error objects.
@Transactional	Manages database transaction boundaries automatically. Opens a transaction before method execution, commits on success, and automatically rolls back if any <code>RuntimeException</code> occurs.	UserServiceImpl, WorkoutServiceImpl, MealServiceImpl, GoalServiceImpl	Guarantees ACID properties. If a database error occurs halfway through saving an entity, the transaction rolls back, preventing corrupt or half-saved data.

3. Spring MVC Request Handling

Annotation	Technical Function	Where Used	Viva / Examiner Tip
@RequestMapping	Maps HTTP requests to controller classes or methods. In FitLog, it sets the base URI prefix for entire controller classes (e.g. <code>@RequestMapping("/api/workouts")</code>).	All 7 Controller classes	<i>Provides clean RESTful path versioning and hierarchy across the application.</i>
@GetMapping	Shortcut annotation for <code>@RequestMapping(method = RequestMethod.GET)</code> . Handles HTTP GET requests to retrieve and read resources.	All 7 Controller classes	<i>Used for all read-only queries (e.g. GET /api/users, GET /api/summaries/daily). It is idempotent and safe.</i>
@PostMapping	Shortcut annotation for <code>@RequestMapping(method = RequestMethod.POST)</code> . Handles HTTP POST requests to create new resources.	UserController, WorkoutController, MealController, GoalController, ChatController	<i>Used when submitting new data to the server (e.g. logging a workout, adding a meal, sending an AI chat message).</i>
@PutMapping	Shortcut annotation for <code>@RequestMapping(method = RequestMethod.PUT)</code> . Handles HTTP PUT requests to update existing resources.	UserController, WorkoutController, MealController, GoalController	<i>Used for updating records (e.g. PUT /api/users/{id} to update weight/height, or PUT /api/workouts/{id} to edit duration).</i>
@DeleteMapping	Shortcut annotation for <code>@RequestMapping(method = RequestMethod.DELETE)</code> . Handles HTTP DELETE requests to remove resources.	UserController, WorkoutController, MealController, GoalController	<i>Permanently deletes a resource by primary key (e.g. DELETE /api/workouts/{id}).</i>
@PathVariable	Extracts dynamic URI path template variables and binds them to method parameters (e.g. <code>/api/users/{id} -> @PathVariable Long id</code>).	All Controller classes	<i>Allows dynamic REST URLs. If someone visits /api/users/5, the value 5 is automatically parsed into the Java Long parameter.</i>
@RequestParam	Extracts query string parameters from the request URL (e.g. <code>/api/summaries/daily?userId=1&date;=2026-09-28 -> @RequestParam Long userId</code>).	WorkoutController, MealController, GoalController, SummaryController	<i>Used for filtering, sorting, or passing auxiliary search criteria that are not primary identity paths.</i>
@RequestBody	Instructs Spring to read the raw HTTP request body (JSON) and deserialize it into a Java DTO object using Jackson ObjectMapper.	UserController, WorkoutController, MealController, GoalController, ChatController	<i>Tell the examiner this is how the browser's JSON data is automatically converted into a strongly typed Java object like WorkoutRequest.</i>
@DateTimeFormat	Declares that a request parameter should be parsed as a date/time using a specific ISO format (e.g. <code>@DateTimeFormat(iso = DateTimeFormat.ISO.DATE)</code>).	WorkoutController, MealController, GoalController, SummaryController	<i>Prevents String-to-Date parsing errors by automatically parsing YYYY-MM-DD into a Java LocalDate instance.</i>
@ExceptionHandler	Designates a method in <code>@RestControllerAdvice</code> to handle a specific exception class or hierarchy.	GlobalExceptionHandler.java	<i>Allows custom handling per exception type (e.g. handling ResourceNotFoundException with HTTP 404 and BusinessException with HTTP 400).</i>

4. JPA & Hibernate ORM

Annotation	Technical Function	Where Used	Viva / Examiner Tip
@Entity	Specifies that the Java class is a persistent entity mapped to a relational database table in MySQL.	User.java, Workout.java, Meal.java, Goal.java	<i>Every row in the database table corresponds to an instance of an @Entity class. It informs Hibernate to track this class for ORM operations.</i>
@Table	Specifies the target table name and database constraints (e.g. <code>@Table(name = "users", uniqueConstraints = ...)</code>).	User.java, Workout.java, Meal.java, Goal.java	<i>Allows the Java class name (User) to map cleanly to a custom database table name (users) with specific database-level indexes.</i>
@Id	Designates the primary key column of the persistent entity.	User.java, Workout.java, Meal.java, Goal.java	<i>Every entity in relational database theory MUST have a unique primary key. @Id specifies which field serves this purpose.</i>

Annotation	Technical Function	Where Used	Viva / Examiner Tip
@GeneratedValue	Configures the generation strategy for primary keys. In FitLog, GenerationType.IDENTITY delegates primary key generation to MySQL AUTO_INCREMENT.	User.java, Workout.java, Meal.java, Goal.java	Ensures that when a new record is saved, MySQL assigns the next available unique integer ID (1, 2, 3...) automatically.
@Column	Customizes database column mapping, including column name, nullable constraints, length limits, and precision.	User.java, Workout.java, Meal.java, Goal.java	Enforces database schema constraints at DDL generation time (e.g. nullable = false, length = 100).
@ManyToOne	Defines a single-valued association to another entity class that has many-to-one multiplicity (e.g. Many workouts belong to One user).	Workout.java, Meal.java, Goal.java	Creates the relational link between child and parent tables. Configured with FetchType.LAZY for optimal database query performance.
@OneToMany	Defines a multi-valued association to child entities that have one-to-many multiplicity (e.g. One user has Many workouts).	User.java	Configured with cascade = CascadeType.ALL and orphanRemoval = true. If a User is deleted, all their workouts, meals, and goals are automatically deleted.
@JoinColumn	Specifies the physical foreign key column used for joining relational entity tables (e.g. @JoinColumn(name = "user_id")).	Workout.java, Meal.java, Goal.java	Creates the user_id foreign key column in the workouts/meals/goals MySQL tables referencing the id column of the users table.
@PrePersist	Specifies a callback method that is automatically triggered immediately before the entity is saved/inserted into the database.	User.java, Workout.java, Meal.java, Goal.java	Used for automatic timestamp auditing. Automatically sets createdAt or loggedAt to LocalDateTime.now() without requiring manual coding.

5. Jakarta Bean Validation

Annotation	Technical Function	Where Used	Viva / Examiner Tip
@Valid	Triggers automatic validation on the annotated object argument before controller method execution starts.	All Controller classes	If any validation rule fails (e.g. negative calories), Spring immediately aborts the request and throws MethodArgumentNotValidException with HTTP 400.
@NotNull	Validates that the annotated field is not null.	GoalRequest, WorkoutRequest, MealRequest, and Entity classes	Prevents NullPointerException by ensuring required fields (like userId, targetWorkoutCount, durationMinutes) must be provided.
@NotBlank	Validates that the annotated character sequence is not null and contains at least one non-whitespace character.	UserRequest, WorkoutRequest, MealRequest, ChatRequest	Stricter than @NotNull. A string with just empty spaces (" ") is rejected. Used for names, food items, and workout types.
@Size	Enforces minimum and maximum boundaries on string length or collection size.	UserRequest.java	Prevents buffer overflow and ensures data sanity (e.g. user name must be between 2 and 100 characters).
@Min	Enforces that the annotated numeric value is greater than or equal to a specified minimum value.	MealRequest.java, Workout.java, Meal.java	Directly implements FitLog's Business Rule 1: @Min(value = 0, message = "Calories must be non-negative").
@Positive	Enforces that the annotated numeric value is strictly positive (greater than 0).	GoalRequest.java, WorkoutRequest.java	Ensures that workout duration and target weekly workout counts cannot be 0 or negative numbers.
@Email	Validates that the string matches the standard RFC compliant email format.	UserRequest.java, User.java	Guarantees that user email inputs contain valid domain structures (e.g. user@domain.com).

6. Jackson JSON Serialization

Annotation	Technical Function	Where Used	Viva / Examiner Tip
@JsonIgnore	Instructs Jackson JSON serializer to completely ignore and omit the annotated property from JSON output.	User.java	Crucial for preventing infinite circular loops when serializing bidirectional JPA entities (User -> Workout -> User -> Workout...).

Annotation	Technical Function	Where Used	Viva / Examiner Tip
@JsonIgnoreProperties	Specifies a list of properties to ignore during JSON serialization or deserialization (e.g. @JsonIgnoreProperties({"hibernateLazyInitializer", "handler"})).	Workout.java, Meal.java, Goal.java	Prevents Hibernate lazy-loading proxy serialization exceptions when Jackson serializes entities fetched from MySQL.

7. Spring Data Queries

Annotation	Technical Function	Where Used	Viva / Examiner Tip
@Query	Defines custom JPQL (Java Persistence Query Language) or native SQL queries directly on repository methods.	WorkoutRepository.java, MealRepository.java	Used in FitLog for complex date-range calorie aggregations (SUM(caloriesBurnt) between startDate and endDate).
@Param	Binds a repository method parameter to a named parameter placeholder in a @Query JPQL string (e.g. :userId, :startDate).	WorkoutRepository.java, MealRepository.java	Prevents SQL injection vulnerabilities by using parameterized query substitution.

8. SpringDoc OpenAPI 3

Annotation	Technical Function	Where Used	Viva / Examiner Tip
@Tag	Categorizes and groups related REST endpoints into logical visual modules on the Swagger UI documentation dashboard.	All 7 Controller classes	Organizes the Swagger page into clean modules: '1. User Management', '2. Workout Logging', 'AI Chatbot', etc.
@Operation	Provides a concise summary and detailed description for a specific REST API endpoint in the OpenAPI documentation.	All 7 Controller classes	Provides human-readable endpoint documentation that examiners see when expanding endpoints on /swagger-ui.html.

9. JUnit 5 & Mockito Testing

Annotation	Technical Function	Where Used	Viva / Examiner Tip
@SpringBootTest	Boots the complete Spring application context in a test environment, loading all beans, repositories, and configurations.	FitLogApplicationTests.java	Used for end-to-end integration testing to verify that the entire Spring context and database connection initialize without errors.
@ExtendWith	Registers third-party test extensions with JUnit 5. In FitLog, @ExtendWith(MockitoExtension.class) initializes Mockito mocks.	WorkoutServiceTest, MealServiceTest, SummaryServiceTest	Enables fast, isolated unit testing without loading the full Spring Boot context or starting MySQL.
@Test	Marks a method as an executable test case that JUnit 5 executes.	All test classes	Identifies test methods. If any assertion fails or an unexpected exception occurs, JUnit reports the test as failed.
@BeforeEach	Specifies that the annotated method must run before each individual @Test method in the test class.	WorkoutServiceTest, MealServiceTest, SummaryServiceTest	Resets mock objects and initializes fresh test data fixtures before each test method runs, ensuring test isolation.
@DisplayName	Provides an expressive, readable name for a test case displayed in IDE and Maven test execution logs.	WorkoutServiceTest, MealServiceTest, SummaryServiceTest	Makes test reports readable for evaluators (e.g. 'Rule 1: Should reject workout when calories burnt is negative').
@Mock	Creates a simulated mock instance of a dependency (e.g. WorkoutRepository) without connecting to the database.	WorkoutServiceTest, MealServiceTest, SummaryServiceTest	Allows testing service logic in pure isolation. We configure mock behavior using Mockito.when(...).
@InjectMocks	Instantiates the tested class and automatically injects all mock dependencies annotated with @Mock into its constructor/fields.	WorkoutServiceTest, MealServiceTest, SummaryServiceTest	Automatically wires mock repositories into WorkoutServiceImpl so the service can be tested directly.

10. Java Language Standard

Annotation	Technical Function	Where Used	Viva / Examiner Tip
@Override	Compiler directive indicating that a method overrides a declaration in a superclass or implements an interface method.	All Service Implementations	<i>Provides compile-time safety. If the interface method signature changes, the Java compiler will immediately catch the mismatch.</i>